

Priority-Aware GPU Co-Scheduling for High Performance Computing

Naman Kulshreshtha
School of Computing
Clemson University
Clemson, SC
nkulshr@clemson.edu

Tapasya Patki
CASC
Lawrence Livermore
National Laboratory
Livermore, CA
patki1@llnl.gov

Aniruddha Marathe
CASC
Lawrence Livermore
National Laboratory
Livermore, CA
marathe1@llnl.gov

Tom Scogland
CASC
Lawrence Livermore
National Laboratory
Livermore, CA
scogland1@llnl.gov

Rong Ge
School of Computing
Clemson University
Clemson, SC
rge@clemson.edu

Abstract—GPU co-scheduling, such as NVIDIA’s Multi-Process Service (MPS), has been proposed as an effective approach to improve resource utilization and system throughput on high-performance computing (HPC) systems. Despite this, its use is limited in production systems due to lack of support for critical features, such as being able to assign varying priorities among the co-scheduled jobs.

In this work, we introduce a novel priority-aware co-scheduling policy, along with performance-bounded co-scheduling, and lay the groundwork for developing these features for future production HPC systems. We focus on a realistic use case, enabling co-scheduling of long-running Large Language Model (LLM) training workloads alongside HPC applications in a performance-bounded and priority-aware manner. We evaluate our proposed approach on a multi-node GPU cluster through actual execution of multiple HPC job queues alongside an LLM training task. Compared to MPS, our priority-aware co-scheduling policy improves the average turnaround time of high-priority jobs by 1.71x for single-GPU experiments and 1.21x for multi-node, multi-GPU experiments.

I. INTRODUCTION

With the emergence of Artificial Intelligence and Machine Learning (AI/ML)-guided scientific simulations, the nature of the mix of jobs running on traditional HPC clusters is changing rapidly. Supercomputers now need to support both traditional bulk-synchronous parallel simulations [1] and AI/ML-guided simulations [2]–[4]. Many supercomputing sites are also deploying in-house, high-availability Large-Language Model (LLM)-based services [5], [6], for use cases ranging from site-specific conversational AIs to code generation and employee education. This has resulted in an emerging trend, where HPC sites need to regularly conduct large-scale online LLM training (and inference) on their HPC systems while still serving traditional HPC user requests.

As a consequence, HPC systems must balance their GPU resource allocations among traditional HPC job requests and long-running AI/LLM job requests to maintain high scientific throughput as well as fairness. This creates significant job scheduling challenges. For example, if an LLM training job, which is typically long-running and resource-hungry, is scheduled to run exclusively on dedicated resources of a large-scale cluster, it can lead to high queue wait times for user-level HPC applications. On the other hand, if shorter-duration HPC

jobs are always picked due to backfilling-based policies, long-running LLM training jobs tend to starve for resources.

A potential approach to mitigate this issue is co-scheduling, allowing HPC jobs and LLM training tasks to concurrently run on shared GPU resources. Modern GPUs support co-scheduling and can be partitioned into multiple smaller GPUs, enabling diverse resource allocation patterns within a node [7]–[9]. However, this capability is rarely utilized in production due to prevailing HPC policies that allocate entire nodes rather than finer-grained resources.

Further, effective job priority management when co-scheduling is necessary for meeting user requirements and Quality of Service (QoS) goals. None of the existing GPU co-scheduling approaches [7]–[9], despite the diversity of options in GPU vendors, expose the dials to effectively support priorities of co-scheduled GPU jobs. Due to lack of low-level tuning dials, existing GPU co-scheduling approaches [10], [11] are limited to optimizing metrics such as average turnaround time (ATT)¹ and makespan², but are unable to support job priorities on a GPU-based HPC cluster.

In this paper, we propose a mechanism to support job prioritization when co-scheduling on GPUs. We also show how priority-aware GPU co-scheduling can be enabled with a production-grade resource manager, Flux [12], and demonstrate multi-node results on a real-world cluster. Resource priority in our research refers to the urgency of obtaining results: high-priority jobs require rapid execution, while lower-priority jobs can tolerate some delay. For example, mission-critical applications such as real-time simulations demand timely results and are classified as high-priority. In contrast, achieving throughput or progress metric over raw performance becomes crucial for lower-priority jobs such as continuous training and inference models and LLMs. We specifically focus on co-scheduling of the real-world use case of training LLMs alongside serving HPC user job requests on NVIDIA GPUs, but our work can be easily applied to support other

¹For N jobs in a queue, where each job i has a wait time of $wait_i$ and execution time of $exec_i$, the average turnaround time is defined as:
$$ATT_{queue} = \frac{\sum_{i=0}^{N-1} (T_{wait_i} + T_{exec_i})}{N}.$$

²Makespan is the total time between the submission of the first job and the completion of the last job in the queue

application use cases (e.g. other long running jobs, or a queue of co-scheduled jobs) as well as to other GPU vendors.

We propose a novel policy called priority-aware co-scheduling (PACS) with bounded slowdown, and compare it with NVIDIA’s default co-scheduling policy as well as a simpler, performance-bounded co-scheduling policy (BCS). Using a real-world job queue of 56 single-node and 20 multi-node jobs, we demonstrate how PACS improves the *ATT* and *makespan* of high-priority jobs, while still achieving high-performance (and throughput for LLM) for low-priority jobs. Our results show that for single-node experiments, PACS improves *ATT* by $1.7\times$ compared to NVIDIA’s default mechanism. Similarly, for multi-node experiments, PACS improves the *ATT* by up to $1.21\times$.

The rest of this paper is organized as follows. We provide a detailed description of existing co-scheduling strategies in Section II. We then present opportunities for co-scheduling in Section III and discuss the limitations of existing approaches in Section IV. Sections V and VI introduce the bounded co-scheduling and priority-aware scheduling policies, and Sections VII and VIII present our evaluation. We present related work in Section IX and conclude the paper in Section X.

II. BACKGROUND

At a high-level, co-scheduling strategies that allow sharing of a GPU can be categorized as *time sharing* or *space sharing*. While we describe specifics on NVIDIA GPUs below (due to the hardware we have access to for evaluation), similar counterparts to co-scheduling exist on AMD GPUs.

Time sharing on GPUs is similar to *time slicing* in multi-tasking operating systems. The GPU scheduler is responsible for maintaining the process and kernel state in memory after each context switch. Time sharing is the NVIDIA’s default mechanism for handling multiple concurrent processes on a GPU.

Space sharing, on the other hand, allows different processes to *physically* share the GPU’s resources. NVIDIA provides two space sharing mechanisms:

- Multi-Process Service (MPS) [7], where the GPU’s compute resources are partitioned but the memory subsystem is shared.
- Multi-Instance GPU (MiG) [8], where both the compute and memory subsystem from the GPU are divided between processes.

Under MPS, compute resources can be set *manually* using the `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` environment variable, which specifies the percentage of Thread Processing Clusters (TPCs) allocated to each process. When this variable is not set explicitly, MPS autonomously handles resource sharing, which we refer to as `default MPS` in this paper.

In `default MPS`, the GPU employs a *leftover policy* for co-scheduling [13], [14], where blocks from different kernels can occupy any slack present in the GPU (where idle streaming multiprocessors (SMs) are detected). This results in improved utilization, but can increase the contention in the L1 cache

and impact the performance of memory-bound applications. Further, as the L1 and L2 cache is shared between processes, MPS can potentially raise security concerns (e.g. possibility of side-channel attacks) at some sites. While MPS allows running workloads from multiple users³, this can result in timesharing of the GPU instead of space sharing.

In order to mitigate these issues, NVIDIA introduced MiG, which allows users to partition a GPU into a *predefined* set of separate GPU instances. This enables physical partitioning of the cache and global memory along with compute resources, which provides resource isolation and guarantees quality of service for each process. MiG allows multiple users to spatially share the GPU concurrently.

MiG partitions are created at the Graphics Processing Cluster (GPC) level, where each partition is allocated at least one complete GPC. Currently, MiG does not support sub-GPC level partitioning, resulting in a relatively coarse-grained and an inflexible partitioning scheme. Another challenge with MiG arises from the fixed partitioning of global memory, where each GPC is assigned to a specific memory controller. For example, on A100 GPUs with two MiG partitions, the available global memory is divided equally. This constraint limits each application to a maximum of 50% of the total GPU memory, which can be limiting if one of the two applications is memory-intensive.

A. Choosing MPS over MiG

In this paper, we consider the case of co-scheduling high-performance applications, which can require significant amounts of resources, including high amounts of memory. As a result, we choose MPS over MiG in our research. This is to support enabling of fine-grained resource allocation and to better utilize the expensive GPU resources. Shared global memory with MPS also proves to be an advantage as it does not constrain applications to only 50% memory like MiG does.

MPS also offers universal compatibility across a diverse set of NVIDIA GPUs, including older GPUs, and allows users to port their code to different HPC systems easily. From the security perspective, we note that users on top-tier HPC systems are strictly vetted. Further, network connections at the site level are highly secure. As a result, side-channel attacks are highly unlikely in these use cases.

III. OPPORTUNITIES FOR CO-SCHEDULING

We begin by outlining the motivation and opportunities for co-scheduling, highlighting both its potential benefits—such as improved resource utilization and reduced job starvation—and presenting a cost-benefit analysis. We identify two major limitations of the `default MPS` co-scheduling approach: (1) its inability to dynamically adapt or conduct application-aware performance tuning, and (2) its lack of support to enforce job priorities among co-scheduled applications.

³By launching multiple MPS servers and setting different paths for each using the `CUDA_MPS_PIPE_DIRECTORY` environment variable

TABLE I: Input parameters for HPC applications

Application	Input
LAMMPS*	-v nx 64 -v ny 64 -v nz 64
Sw4lite*	Problem Loh.1 from SCEC test suite, t=360
Laghos*	-pt {task-partition}, -m {input-mesh}, -rp 2, -tf 0.6
Quicksilver*	Base mesh size of 16 per process with 30 particles per mesh. for 8 tasks (2x2x2 topology), global mesh is 2x16 in each dimension with a total of 30 particles per mesh block.
Kripke*	--zones {24 per process}, --groups 64, --gset 1. for 8 tasks (2x2x2 topology), global zones is 2x24 in each dimension
GEMM	--repfact 150, --sizefact 35
Energy	--repfact 150, --sizefact 70
ADI	--repfact 125, --sizefact 150
STREAM	--repfact 75, --sizefact 200
Pressure	--repfact 75, --sizefact 95

* Indicates multi-node experiments.

A. Experimental Setup

We first describe our experimental setup to compare default MPS and timesharing on both single-node and multi-node applications. Single node experiments were conducted on a node with two Intel Xeon Skylake CPUs, each with 20 cores and 370 GB of DRAM. Each node included two V100 GPUs with 16GB of VRAM, connected via PCIe. Multi-node experiments were conducted on a cluster with 4 nodes, with the same CPU architecture and memory, but using a 32GB variant of the GPU. Node-to-node communication was facilitated by a 25 Gbps InfiniBand interconnect. Applications were compiled using CUDA 12.2 and GCC 12.2.1.

The experimental setup described in this section also applies to upcoming sections in the paper. All experiments used a production-grade resource manager, Flux [12]. Flux is an open-source resource management framework developed by Lawrence Livermore National Laboratory (LLNL) and is deployed on several production HPC systems. Unlike many GPU co-scheduling and policy design papers [15]–[17], the results presented in this paper are not limited to single-node configurations, and are neither simulated nor emulated.

B. Applications

We evaluate a total of 10 HPC applications, consisting of five mini/proxy applications, three Polybench kernels, and two RajaPerf [18] applications in our experiments. The applications and their respective inputs are listed in Table I. LAMMPS employs the ML-SNAP package [19] to perform Molecular Dynamics (MD) simulations. SW4Lite [20], designed for seismic wave propagation simulations. Laghos [21] implements partial differential equation solvers for compressible shock hydrocodes; we use a cubic mesh file provided in the source distribution. Quicksilver [22] is a proxy application that solves the dynamic Monte Carlo particle transport problem. Kripke [23] is a scalable 3D Sn deterministic particle transport code.

Application	Single-node GPU Utilization (%)	Multi (4)-node GPU Utilization (%)
Kripke	44.06	42.18
Laghos	91.23	41.20
LAMMPS	99.14	76.66
QuickSilver	37.19	32.12
SW4Lite	99.26	84.30

TABLE II: Average GPU utilization for HPC applications.

From the benchmark suites, we employ the RajaPerf implementations of the Polybench GEMM, STREAM and ADI kernels, along with two RajaPerf applications: Pressure and Energy.

For the long-running LLM training job, we utilize LLAMA-Factory [24] for instruction fine-tuning with Meta’s LLama3-7B [25] model. We focus on fine-tuning as it exhibits similar computational patterns to pre-training (gradient backpropagation through billions of parameters) while being tractable for experimental setup. Checkpoints are saved every 50 iterations, with each checkpoint requiring approximately 1.2 seconds to write and 7 seconds to restore on average. We conduct experiments on two datasets: (1) a small LLAMA-Factory dataset comprising 100 samples trained for 80 epochs, and (2) a 1,000-sample subset of the Alpaca dataset [26] trained for 2 epochs. On single node experiments, the smaller dataset takes 41 minutes and the larger one takes 4.5 hours. We use the small dataset in Section IV-A and the larger dataset for all other experiments. For multi-node configurations, we use 2xV100 GPUs with a batch size of 4 and checkpoint every 20 iterations for a 1000 epoch run.

C. Observation 1: Exclusive allocations underutilize GPUs

In order to analyze GPU utilization under exclusive (serial) allocations—scenarios we refer to as “serial” henceforth, we measured GPU utilization using NVIDIA’s `nvidia-smi` tool. This tool reports the percentage of time the GPU is actively executing kernels during each sampling interval. We opted for `nvidia-smi` because `nsight-compute` [27] only very recently added support for MPS when measuring actual SM utilization. Table II presents the average GPU utilization for the HPC applications in our study.

Our evaluation highlights that, with the exception of GEMM (not shown in table), applications do not fully utilize the available GPU resources, revealing significant opportunities for improved system throughput via co-scheduling. HPC applications such as Kripke and Quicksilver have particularly low GPU utilization, both in single-node and multi-node runs, where the GPU either idles or waits for memory operations to complete. Overall GPU utilization drops even further in multi-node runs due to the additional overhead of communication and synchronization between processes across nodes. For example, for Laghos, the average GPU utilization drops from 91.2% in the single-node execution to 41.2% in the multi-node execution.

Opportunity: *Can co-scheduling improve system utilization and throughput?*

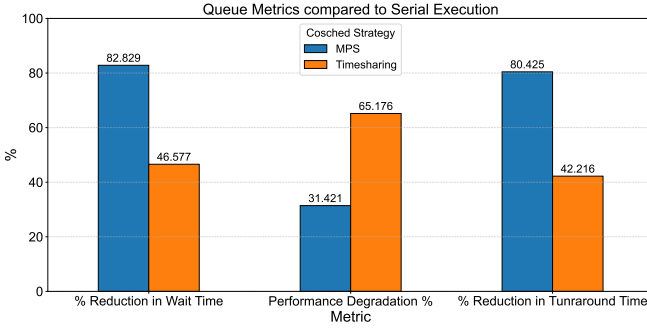


Fig. 1: Comparison of average wait, execution, and turnaround times of default MPS and timesharing with a queue of 56 jobs.

D. Observation 2: Co-scheduling improves turnaround times

We evaluated the average turnaround times for a trace of 56 HPC applications submitted alongside an LLM training job, with HPC job arrivals following Poisson distribution (see Section VII). Our analysis shows that in exclusively allocated execution, long-running jobs such as LLM training can significantly delay HPC jobs, leading to severe resource starvation and increased queue wait times.

Opportunity: *Co-scheduling enables earlier job starts by trading some performance for reduced wait times.*

Figure 1 compares the reduction in wait times, degradation in performance and turnaround times of default MPS and timesharing techniques relative to the serial run. We observed that MPS results in 82.8% reduction in average wait time while trading off 31% performance, compared to the serial run. This results in 80.4% reduction in average turnaround time for MPS compared to serial run. Note that default MPS consistently performs better than timesharing, so in the rest of this paper, we will omit comparison with timesharing.

E. Observation 3: Application-specific tuning is needed

Application-specific tuning of MPS can enable more efficient resource sharing and improved overall throughput. When co-scheduling, the resulting performance degradation depends on the application’s resource usage profile. Compute-bound applications exhibit a proportional decrease based on SMs allocated on the GPU. In contrast, memory-bound applications maintain stable performance until SM allocation drops below a certain application-specific threshold, after which performance declines sharply as the workload shifts from relatively being memory-bound to compute-bound [11].

Figure 2 presents a performance comparison of two HPC applications: SW4Lite, which is compute-bound, and Stream [28], which is memory-bound. The figure compares their speedup at three different MPS percentages (80, 60 and 30) and for each MPS% compare the performance of running the application in-isolation at that MPS% vs. co-scheduled with the LLM. Performance below 1.0 is worse than the individual application’s serial run.

SW4Lite exhibits a near-linear performance decline from 100% to 30% MPS and we see no impact on performance when co-scheduled with LLM. In contrast, STREAM shows

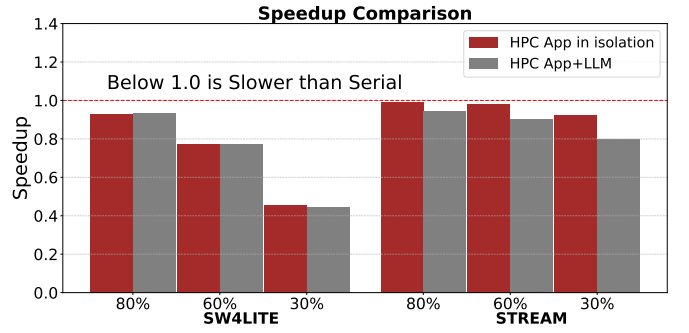


Fig. 2: Performance of applications with varying MPS percentages set manually, with and without co-scheduling.

only a minor performance degradation from 80% to 30% MPS since reducing SM allocation does not significantly impact memory bandwidth utilization. However, when a memory-bound HPC application is co-scheduled with LLM, contention on shared resources such as DRAM and L2 cache leads to performance degradation for both applications. This effect is particularly evident with Stream, which suffers greater performance loss compared to SW4Lite co-scheduled with LLM. Notably, compute-bound applications do not experience significant degradation when co-scheduled with LLM as the basic unit of compute work a “thread block” can run independently on each SM and hence have no contention issues, minimizing interference.

Opportunity: *Application-specific MPS fine tuning can drive significant performance gains when co-scheduling.*

IV. LIMITATIONS OF DEFAULT CO-SCHEDULING

We present the main limitations of existing co-scheduling approaches in the subsections below.

A. Static Resource Allocation

None of the existing co-scheduling strategies provide the ability to *dynamically* allocate portions of the GPU to individual processes *explicitly* during application execution. The leftover policy from default MPS cannot be controlled directly by a user or an administrator. Partitioning is thus static, and once assigned, GPU resources cannot be dynamically reallocated. This limitation greatly impacts performance of co-scheduled applications, and also results in wasted compute capacity when one process finishes earlier than another.

To illustrate this issue, we devised a MPS-based co-scheduling experiment with two applications, Laghos and an LLM training application Laghos was allocated 80% of the GPU, while the LLM job received 20%. When co-scheduled, Laghos took 246 seconds (serial baseline of 230 seconds). However, after Laghos finished, the LLM job continued to run with only 20% of the GPU, and took 119 minutes to complete. For 115 of those minutes, 80% of the GPU remained underutilized. In contrast, when the same LLM job was allocate 100% GPU access, it completed in 41 minutes.

In order to address this vendor limitation that is outside our control, we employ a dynamic system using checkpointing

and restart (C/R) to adjust resource allocations, which is a significant engineering effort. This approach allows MPS resource allocations to be modified at runtime. We refer to this approach as MPS+C/R in this paper. We leverage the signaling capabilities of Flux Resource Manager [12] to enable this mechanism. Each job runs within a Flux client, which communicates with the scheduler and handles restarts when resource reallocation is required. Upon job completion or the arrival of a new job, the scheduler recalculates resource allocations for all active jobs and signals the client to restart them with the updated resource configuration. This mechanism can be applied to multiple co-scheduled jobs and is not limited to only two co-scheduled processes.

B. No support for prioritization

Existing co-scheduling strategies also assume that all jobs running simultaneously on a GPU have the same *priority*. This turns out to be a major limitation for supercomputing sites. User-level prioritization of jobs is a necessary part of batch scheduling on HPC clusters, as these systems serve diverse sets of users—ranging from those executing mission-critical science applications to academic collaborators and students. Multi-factor priority plugins to support different classes of users are thus in effect across many sites. The inability to conduct priority-aware co-scheduling is one of the key reasons for underutilization of expensive GPU resources.

This work addresses this limitation by developing priority-aware co-scheduling strategies that minimize average turnaround time in HPC environments. We specifically examine scenarios where a long-running job monopolizes cluster resources, blocking the queue and increasing wait times for subsequent jobs. Our approach employs co-scheduling to enable simultaneous execution of the long-running job alongside other queued jobs, thereby allowing the former to progress while reducing wait times for the latter. Critically, we investigate methods to maintain optimal performance for high-priority jobs and ensure bounded slowdown for low-priority jobs in the HPC queue, while still enabling progress on the long-running workload, such as an LLM training workload.

V. PERFORMANCE-BOUNDED CO-SCHEDULING

We first propose a simple priority-aware, performance bounded co-scheduling approach, referred to as Bounded Co-scheduling Strategy (BCS). BCS serves as a baseline policy for priority-aware co-scheduling for the rest of the paper.

When co-scheduling, BCS prioritizes the HPC application to ensure predictable performance while still allowing the LLM application to make progress. The key idea behind BCS is to allow users to specify a maximum acceptable slowdown $s\%$ for their HPC applications, and to meet this constraint when deciding GPU resource allocation at runtime. For the LLM application, BCS does not make performance guarantees, but allows it to make continual progress by always assigning the maximum possible GPU resources without slowing down the HPC application. BCS achieves this dynamic allocation by

leveraging the MPS+C/R mechanism to enable LLM applications to dynamically resize.

Whenever a new application arrives, the LLM application is restarted with an updated resource allocation (done through MPS+C/R and co-scheduled alongside with the incoming application, this results in no overhead for the HPC applications. We further discuss the impact of C/R on LLM performance in Section VIII-E. Note that while we depict examples utilizing an HPC application (and later in the paper, a queue of HPC jobs) co-scheduled with a long-running LLM application, the ideas and results from this paper generalize to any co-scheduled workloads involving application priorities.

BCS leverages empirical knowledge of application behavior to determine the appropriate GPU share (via. MPS%) that satisfies the user-defined bound on performance slowdown. For the scope of this paper, we pre-compute the mapping for each HPC application based on single-node runs and store as a lookup-table. Developing portable analytical performance models or heuristics to determine such mappings can be challenging due to application complexity [10], resources allocated and interference from other co-scheduled application. Addressing these challenges requires a model that can identify how much percentage of total time is spent on pure compute, determine the relationship between individual kernels and resources allocated, and quantify the slowdown due to the second application—capabilities. We plan to develop it in a future work.

BCS uses these precomputed MPS% settings to co-schedule HPC jobs alongside LLM, ensuring that the HPC application's performance degradation remains within the specified bound. For single-node execution, this guarantee is strict. For multi-node jobs, we rely on best effort due to the variation observed from inter-node synchronization differences.

A. Determining Application Performance Bounds

Table III lists the applications evaluated in this work for both single- and multi-node experiments, along with the MPS% which resulted in ($s\%$) of 10%. Note that we use the 10% slowdown as a use case, and the same methodology can be used for any user-defined slowdown constraint. We divide the applications into two categories:

- **Compute-bound jobs:** These include LAMMPS, SW4Lite, and Laghos, all of which consist of compute-heavy kernels. These applications need a high MPS% to keep the slowdown under 10%. Quicksilver, while also being compute-bound, experiences significant memory transfer overhead, enabling lower MPS% allocation while satisfying the 10% slowdown constraint.
- **Memory-bound jobs:** These include RajaPerf kernels, which saturate the GPU memory bandwidth and quickly hit a performance plateau. Kripke behaves similarly due to its high memory transfer time, and is thus grouped with memory-bound applications.

TABLE III: MPS allocations to ensure application slowdown under 10%.

Compute-Bound (High MPS Dependence)		Memory-Bound (Low MPS Dependence)	
Application	MPS%	Application	MPS%
LAMMPS*	85	Kripke*	50
SW4Lite*	75	Energy	60
Laghos*	80	ADI	60
GEMM	85	STREAM	60
Quicksilver*	70	PRESSURE	60

* Indicates multi-node applications.

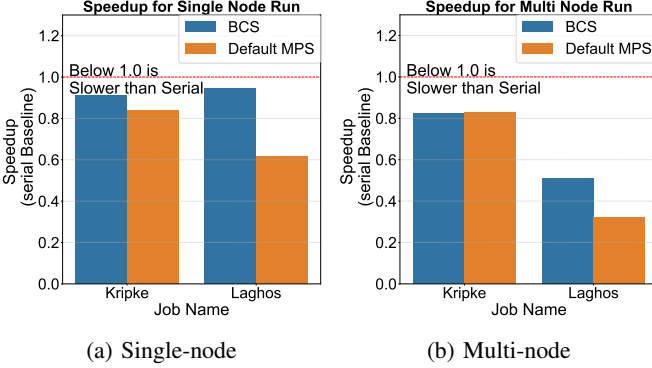


Fig. 3: HPC application performance when co-scheduled with LLM in single- and multi-node configurations. Performance is normalized to the serial run of each application.

B. Co-scheduling Results with BCS

Figures 3a and 3b show the speedup of *Laghos* (compute-bound) and *Kripke* (memory-bound) when co-scheduled with LLM in single-node and a 4-node configurations, and compares these to default MPS.

For *Laghos*, in the single-node configuration, BCS improves the HPC application’s performance by 50% when compared to default MPS. For the multi-node configuration, this improvement is 40%. For *Kripke*, the performance does not change significantly between BCS and default MPS. For both single- and multi-node setups, we observe that BCS either matches the performance (for memory-bound applications) or outperforms default MPS (for compute-bound applications).

We also observe that the overall performance degrades significantly for *Laghos* between single-node and multi-node configurations when co-scheduling, with a slowdown of up to $2x$ for multi-node configuration for both BCS and default MPS. This is primarily due to synchronization issues, as slowdown on a single GPU can introduce a delays for the entire application. We collected event traces for both the serial and BCS runs on the multi-node system using NVIDIA Nsight Systems [29], and observed that the time spent on communication increased by $2x$. compared to the serial run—rising from 25% to 54% of the total execution time.

C. Limitations of BCS

In the previous sections, we explored 2 jobs being co-scheduled (one HPC application and an LLM). In practice, and for the rest of this paper, we focus on job queues with tens of HPC jobs that are executed along with the LLM (not simulated or emulated). When presented with a job queue of HPC applications, BCS allocates equal priority to all the HPC applications, allowing each of them to be co-scheduled with the LLM with an MPS percentage with a corresponding slowdown of $s\%$. As a result, it cannot address HPC applications with different priorities, where high-priority jobs may want to execute at full capacity with zero slowdown. We address this limitation in the next subsection.

VI. PRIORITY-AWARE CO-SCHEDULING

This section describes our approach for designing a priority aware co-scheduling strategy for HPC applications. To handle jobs with different resource requirements based on their priority, we propose a co-scheduling strategy that allocates more resources to high-priority jobs while ensuring a predictable slowdown for low-priority jobs. In this approach, high-priority jobs are granted a larger share of GPU resources—potentially the entire GPU—whereas low-priority jobs receive a controlled fraction to ensure their slowdown does not exceed a user-defined threshold s . The algorithm implementing this policy is presented in Algorithm 1. Let Q_N denote a job queue with N jobs, where each job has a submission time T_{submit_i} and a target slowdown factor s . We assume that at most two jobs may run concurrently: one user-submitted job and an LLM job. New jobs must wait in the queue until resources become available. Once a job is ready to be scheduled, the amount of GPU resources allocated to it, denoted R_{J_i} , are determined based on its priority and the slowdown constraint s :

- **High-priority HPC jobs:** These jobs receive exclusive access to GPU resources and are allocated 100% of the GPU. Any running job (e.g. the LLM) is paused until the high-priority job completes.
- **Low-priority HPC jobs:** These jobs can be co-scheduled with the LLM but must adhere to the user-defined slowdown tolerance s . The GPU allocation R_{J_i} is selected from the pre-profiled lookup table, as with BCS.
- **LLM job:** The LLM job is treated as the lowest priority job. It either runs when there are idle resources, or when it can share resources with a low-priority HPC job. Its resource share is dynamically adjusted using $\text{MPS} + C/R$.

the LLM does C/R only when the new resources count is different from the previous one. Currently we only support two level of priorities but this could be easily extended to support multiple by having different slowdown factor for each priority level.

VII. EVALUATION

We evaluate three co-scheduling policies: default MPS, bounded co-scheduling (BCS), and priority-aware co-scheduling (PACS) using the *serial run* as a baseline. In the serial run, jobs run sequentially with exclusive GPU access.

Algorithm 1 Priority-Aware Co-Scheduling Algorithm

Require: Job queue Q_N with N jobs

Require: Each job J_i has:

Submission time T_{submit_i} ,
Priority $P_i \in \{P_{\text{high}}, P_{\text{low}}\}$,
Slowdown tolerance s_i
Resource Allocation R_i

Require: Set of all GPUs $G = \{G_1, G_2, \dots, G_M\}$

```
1: while true do
2:   if any GPU  $G_k \in G$  has no user job running then
3:     Run LLM on all idle GPUs with 100% MPS
4:   end if
5:   if a new job  $J_i$  arrives at  $T_{\text{submit}_i}$  then
6:     Extract  $P_i$  and  $s_i$ 
7:     Check available free GPUs  $G_{\text{free}} \subseteq G$ 
8:     if not enough GPUs available for  $J_i$  then
9:       Wait until resources become available
10:    end if
11:    if  $P_i = P_{\text{high}}$  then
12:       $R_{J_i} \leftarrow 100$ 
13:    else
14:      Lookup  $R_{J_i}$  from BCS data, such that  $\leq s_i$ 
15:    end if
16:    Allocate GPU and Configure (MPS+C/R):
17:    Select subset  $G_{J_i} \subseteq G_{\text{free}}$  for  $J_i$ 
18:    for each GPU  $G_k \in G_{J_i}$  do
19:      if LLM is running on  $G_k$  then
20:        if  $P_i = P_{\text{high}}$  then
21:          Pause LLM on  $G_k$ 
22:        else
23:          Set  $R_{\text{LLM}} \leftarrow 100 - R_{J_i}$ 
24:          Restart LLM on  $G_k$  with  $R_{\text{LLM}}$ 
25:        end if
26:      end if
27:    end for
28:    Allocate  $R_{J_i}$  on  $G_{J_i}$ 
29:    Launch job  $J_i$  across  $G_{J_i}$ 
30:  end if
31: end while
```

A job queue holds incoming jobs sorted by submit time, with each job specifying requested nodes, duration, and priority. The scheduler manages this queue without preemption, allocating more resources to higher-priority jobs during co-scheduling. The objective is to improve high-priority job performance without compromising overall queue performance. We quantify co-scheduling effectiveness using average turnaround time (ATT) and makespan as key metrics.

For the results presented in this paper, for simplicity of results we assume a maximum performance slowdown of $s = 10$ for all the applications but this is not limited to one value and can be defined separately for each application by the user. The associated MPS% values for $s = 10$ are listed in

Table III. default MPS was described in Section II. Note that default MPS is not priority-aware and thus not a good baseline for priority-awareness. However, we present its results for completeness.

For single-node experiments, we use one node with a single GPU, resulting in $G_k = 1$ in Algorithm 1. Each job requests one task with one GPU per task. For multi-node experiments, we run on 4 nodes with 2 GPUs per node, for a total of $G_k = 8$ GPUs. In this setting, each job requests 8 tasks, with one GPU per task. At present, varying number of GPUs per job is an engineering challenge as the current implementation in Flux is experimental and entirely managed by the user. Future extensions enabling Flux to natively manage GPU co-scheduling across jobs are underway. Addressing this limitation is an engineering issue and does not significantly impact the results presented in this paper.

Job submission times follow a Poisson distribution with a mean arrival rate of $\lambda = 8$, we selected an arrival rate of 8 jobs/hour to maintain sustained queue pressure throughout our experiments, ensuring the scheduler always had pending jobs to make co-scheduling decisions—representative of busy production HPC clusters. We evaluate three HPC queue types based on application characteristics, as listed in Table III.

- **Compute-bound:** Composed entirely of compute-intensive applications,
- **Memory-bound:** Includes only memory-bound applications and
- **Mixed:** A balanced combination of compute- and memory-bound applications drawn from the two categories above.

For single-node experiments, each queue contains 56 HPC jobs. Application runtimes have been adjusted to approximately 5 minutes by tuning the number of iterations, which does not impact their compute or memory characteristics. For multi-node experiments, each queue contains 20 HPC jobs selected from the applications marked with * in Table III. We exclude RajaPerf applications from the multi-node experiments, as they are primarily designed for single-process execution and launch independent GPU kernels without inter-process synchronization.

In each case, HPC jobs are evenly split between high- and low-priority classes. We refer to the *high-priority queue* as the subset of high-priority jobs extracted from the trace, and to the *mixed-priority queue* as the complete job queue containing both high- and low-priority HPC jobs.

VIII. RESULTS

In this section, we first evaluate the effectiveness of PACS for job queues with different characteristics and priorities on single-node jobs. Next, we present the results for multi-node jobs. We then discuss GPU utilization, before finally presenting a practical trade-off between the training progress for the LLM job and the performance of HPC applications.

A. Single-Node Results with PACS

We compare the proposed co-scheduling policies using two metrics: (1) ATT, and (2) Makespan. We normalize both

metrics by calculating the ATT $\text{Speedup} = \frac{ATT_{\text{Serial}}}{ATT_{\text{CoSched}}}$ and the Makespan $\text{Speedup} = \frac{Makespan_{\text{Serial}}}{Makespan_{\text{CoSched}}}$. We categorize the results by the type of HPC jobs in the queue, namely, compute-bound, memory-bound and mixed workloads; and further discuss the different priority levels for the HPC jobs.

Figure 4 shows the comparison of ATT between default MPS, PACS, and BCS for different workload queue types (compute-bound, memory-bound and mixed workloads), and for both high-priority and mixed-priority combinations. Due to small statistical variance we are omitting the error bars from this and all subsequent figures. Note that the focus here is on the HPC jobs in the queue, so the LLM job’s turnaround time is excluded in this figure. The first group plot shows speedups for high-priority jobs, while the second group shows the speedups for mixed (i.e., both high and low) priorities. We discuss these in detail below.

1) *Compute-bound Workloads*: For compute bound jobs, as shown in Figure 4, PACS yields the highest speedup, achieving a 7.42x improvement over the serial baseline and a 2.38x speedup over default MPS, primarily because high-priority HPC jobs are allocated all available SMs. In contrast, BCS—which treats all HPC jobs in the queue as having equal priority with a bounded slowdown—achieves a 5.65x and a 1.8x speedup over serial and default MPS, respectively.

The second group in Figure 4 shows results for the mixed-priorities. Here, for compute-intensive jobs, PACS achieves a 7.34x speedup over the serial baseline and a 2.7x over default MPS. Similar to the high-priority results, this improvement stems from intelligent allocation for both high-priority jobs (which are granted full GPU access) as well as low-priority jobs (which still receive better GPU allocation than under default MPS).

Figure 5 shows a deeper dive for the low-priority jobs only from the mixed-priority queue shown in Figure 4, focusing on individual application execution times. The y-axis here shows the normalized execution time, or slowdown, compared to the serial run. Note that both BCS and PACS bound this slowdown to less than 10%. Here, we observe that applications such as Laghos and LAMMPS experience severe performance degradation under default MPS, where their relatively infrequent kernels (fewer than 100,000) are interleaved with the LLM’s massively parallel kernels (on the order of tens of millions). As a result, HPC applications receive less GPU time and make slower progress. These applications benefit significantly from performance-bounded sharing under BCS, which improves their performance by up to 1.3x over default MPS. Other applications that launch fewer than 10,000 kernels show more modest speedup of up to 1.1x.

Figure 5 also shows the slowdown for the LLM. For LLMs, the best performance is achieved under default MPS, which schedules kernels in a First-Come, First-Served (FCFS) manner and maximizes GPU pipeline occupancy. As expected, because the LLM job is treated as the lowest priority task, it suffers higher performance degradation with BCS and PACS, which are designed to prioritize HPC workloads.

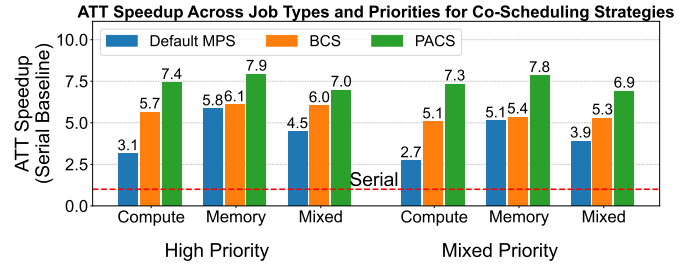


Fig. 4: ATT speedup for single-node experiments, showing high-priority and mixed-priority jobs.

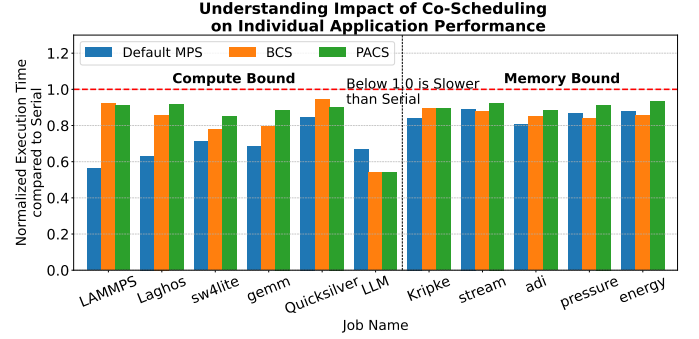


Fig. 5: Individual application slowdowns for low priority jobs only in the mixed-priority queue.

Table IV shows the makespan results for the HPC jobs (HPC-only) as well as for the entire job queue (HPC+LLM). For compute-bound HPC applications in the queue, we observe a 1.78x improvement over serial for PACS and 1.44x over default MPS. When we include the LLM job, the speedup for makespan in both PACS and BCS diminishes, and default MPS delivers overall best makespan. Note that this is expected, as BCS and PACS prioritize the HPC tasks. Also note that default MPS is not priority-aware, so while we show this result for completeness, it is not a fair comparison in terms of priority-aware co-scheduling.

Job Type	HPC Only			HPC+LLM		
	Default MPS	BCS	PACS	Default MPS	BCS	PACS
Compute	1.26	1.67	1.78	1.24	1.01	1.0
Memory	1.56	1.62	1.8	1.12	1.03	1.0
Balanced	1.45	1.61	1.74	1.28	0.98	0.99

TABLE IV: Normalized makespan speedup under different co-scheduling strategies.

2) *Memory-bound Workloads*: Looking at memory-bound jobs in Figure 4, PACS achieves a 1.36x speedup over default MPS, while BCS offers negligible speedups. This trend is further illustrated in Figure 5, where memory-bound applications such as *Stream*, *ADI*, and *Pressure* exhibit similar execution times for both default MPS and BCS, resulting in similar ATT. This highlights that not all applications benefit from exclusive GPU access: memory-bound kernels are limited by memory bandwidth rather than compute throughput.

Figure 4 also shows memory-bound jobs in the mixed-priority queue. Here, PACS achieves a 1.53x ATT speedup over default MPS, whereas BCS shows similar speedup to default MPS. Moreover, default MPS performs rel-

atively well due to reduced kernel-level contention: memory-bound jobs launch fewer kernels, leading to less interference from concurrently running LLM workloads.

In terms of makespan (Table IV), the trend is similar to that of compute-bound makespan speedups. For HPC-only jobs, both PACS and BCS outperform default MPS. For the full queue (HPC+LLM), default MPS achieves the best makespan, but is not priority-aware.

3) *Mixed Workloads (compute and memory-bound)*: Here, for the high-priority jobs in the queue, PACS achieves an ATT speedup of $1.56x$ over default MPS. For the mixed-priority queue, PACS achieves a $1.7x$ speedup over default MPS. For both priority queues the improvement in speedup is due to compute-bound applications benefiting from full GPU access, which reduces their execution time and improves the overall ATT. This effect is also visible for default MPS which performs better relative to the memory-bound jobs, where resource availability has limited impact on execution time.

In terms of makespan, similar to previous results, for HPC-only jobs, both BCS and PACS outperform default MPS. When considering the full HPC+LLM queue, default MPS achieves the best makespan, but is not priority-aware.

B. Multi-Node Results with PACS

Table V presents the ATT improvements for both high-priority and mixed-priority jobs in the queue in a multi-node setting, with respect to the serial baseline. For the high-priority jobs, PACS outperforms BCS and default MPS by $1.1x$ and $1.26x$, respectively. For the mixed-priority jobs, PACS provides improvements of $1.06x$ over BCS and $1.21x$ over default MPS. Compared to single-node results, PACS clearly outperforms default MPS. However, in the multi-node setting, this advantage diminishes as node-to-node variability introduced by varying amount of memory contention causes uneven GPU slowdowns (stragglers); this, in turn, increases cross-node communication and synchronization overhead. We observed this behavior with Laghos (see Section V-B), which explains the reduced gains in multi-node co-scheduling. Memory-bound jobs maintain similar performance across both configurations, as their bottlenecks are unaffected by inter-node synchronization.

Makespan evaluation with multi-node configuration follows single-node trends. We are omitting these results due to space constraints.

Job Type	Default MPS		BCS		PACS	
	High Prio.	Mixed Prio.	High Prio.	Mixed Prio.	High Prio.	Mixed Prio.
Multi-Node	8.58	8.88	9.89	10.16	10.83	10.73

TABLE V: ATT speedup in multi-node configurations.

C. GPU Utilization Behavior

We examine the GPU utilization to understand runtime behavior. When co-scheduling, the GPU is nearly always active due to the high kernel launch frequency of LLM workload. As a result, `nvidia-smi` consistently reports near

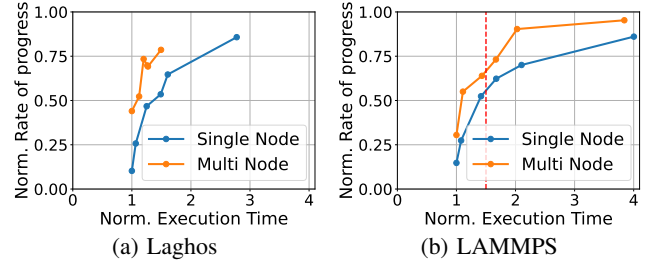


Fig. 6: Trade-off between rate of progress of LLMs versus performance of co-scheduled HPC applications.

100% utilization across all strategies—even in cases where application-level performance differs significantly. This is a significant improvement over the utilization numbers that were reported in Table II.

D. Understanding LLM Progress and HPC Performance

We now present a detailed analysis of how performance is traded between the LLM and the HPC applications, for both single-node and multi-node configurations. Figure 6 shows this trade-off for two HPC applications (Laghos and LAMMPS) when co-scheduled with the LLM training. Each data point in the figure represents a unique MPS allocation where P% MPS is allocated to the HPC application and Q% MPS is allocated to LLM training such that $P+Q=100\%$. Values used for P (left to right on the graph are): 90%, 80%, 60%, 50%, 40% and 20%. These are not shown in the figure for readability. Normalized HPC application execution time with respect to the fastest MPS configuration (100%) is shown on the x-axis. Similarly, the y-axis shows normalized iterations per second of the LLM training with respect to 100% MPS performance. We refer to the LLM performance as the *rate of progress* in the remainder of this discussion, as it is reported per-iteration.

Note that the data points for 100% MPS are not shown in Figure 6, as these correspond to a serial execution and do not represent a configuration that is co-scheduled. Steeper slopes in the figure imply significant gain in the rate of progress for LLM training without significant loss of performance of the co-scheduled HPC application. On the other hand, flatter slopes imply larger degradation in the co-scheduled HPC application’s performance without much loss in LLM training’s rate of progress.

Figure 6 (a) shows the trade-off for Laghos with LLM training, and Figure 6 (b) shows the trade-off between LAMMPS. Both single-node and multi-node results are presented for the six MPS% values mentioned earlier (leftmost data point corresponds to 90% MPS for the HPC application, and rightmost data point corresponds to 20% MPS). We first observe that co-scheduling the LLM with Laghos is better in terms of rate of progress for the LLM when compared to co-scheduling with LAMMPS, as LAMMPS is more MPS-sensitive than Laghos. This is also explained by the larger performance degradation seen with LAMMPS with decreasing MPS percentage (left to right on the x-axis). On the right hand side of the dotted red line, the rate of progress of LLM improves from 0.7 to 0.95 (25%) at the cost of more than

2.66x slowdown (1.2 to 2) in normalized execution time of the application. With *Laghos*, we also see that the multi-node results indicate less degradation in terms of rate of progress compared to the single node results.

E. Cost of Checkpointing Restart

Since our dynamic resource allocation mechanism relies on LLM checkpoint/restart (C/R) capabilities, we evaluate the associated overhead across two representative configurations. For 7B Model (Single V100 GPU) used in the current work each checkpoint incurs a cost of approximately 1.3 seconds. Over 6,790 training iterations with checkpointing every 50 iterations, this yields a cumulative checkpointing time of 138 seconds. Each restart encompassing the reload of model weights and optimizer state from disk costs approximately 7 seconds; across 54 restarts, this totals 378 seconds. The combined C/R overhead of 516 seconds represents less than 3% of the total training runtime of 17,435 seconds. As C/R only effects the LLM performance, the ATT of HPC applications remains unaffected.

For a larger model, Llama3.1-70B ($2 \times$ A100 GPUs), the per-checkpoint cost rises to 122 seconds, yielding a cumulative checkpointing time of 14,191 seconds over 5,816 training iterations. With an average restart cost of 100 seconds across 54 restarts, the total restart overhead amounts to 5,400 seconds. Together, the combined C/R overhead of 19,591 seconds constitutes approximately 14% of the total training runtime of 139,584 seconds. While the absolute checkpoint cost grows with model scale, this overhead can be mitigated by triggering checkpoints only upon the arrival of a new application in the queue, or by reducing the checkpointing frequency. Furthermore, replacing PyTorch’s native checkpointing with more efficient strategies, such as asynchronous or incremental checkpointing [30], [31], can reduce this overhead.

IX. RELATED WORK

On modern heterogeneous CPU-GPU systems, technologies such as Nvidia’s Multi-Process Service (MPS) and Multi-Instance GPU (MIG) have been widely studied to enable and optimize concurrent execution of diverse workloads. Bashizade et al. presented early exploration on an adaptive, GPU kernel-aware multi-tenancy approach to drive system throughput using MPS requiring source-to-source transformations of HPC codes [32]. Saroliya et al. presented a reinforcement learning-driven approach for selecting and co-scheduling GPU workloads using MIG and MPS [10]. Pavlidakis et al. presented an intermediate code-level memory and compute isolation approach for Nvidia GPUs [33]. Bakita et al. presented an effective, user-level alternative to MPS for SM-level partitioning of Nvidia GPUs [34]. Several important approaches based on MPS and MIG have been presented specifically for co-scheduling deep learning workloads [35]–[37]. While these approaches provide a foundation for efficient GPU co-scheduling, they either do not support priority-aware co-scheduling, lack general applicability for HPC-ML work-

loads, often require significant manual tuning, and may not adapt well to dynamic workload changes.

Several works have investigated priority-aware scheduling across GPU partitions using Nvidia’s MIG technology [38], as well as priority-based kernel scheduling within partitions using MPS [39]. Priority-aware scheduling for real-time GPU systems has also been explored, using MPS and task preemption mechanisms [40]. Liu et al. demonstrated the deployment of such priority-aware scheduling mechanisms in production-level clusters for deep-learning workflows [15], while Babaei et al. proposed spatio-temporal co-scheduling of inferencing tasks with priority awareness [41]. Our work addresses the limitations of these prior approaches by targeting general HPC-ML workloads thereby providing broader applicability.

Coarse-grained GPU co-scheduling approaches to improve system resource utilization and task completion rate have been explored for deep learning (DL) workloads [42]. Efficient execution of services such as simultaneous GPU query processing through concurrent execution has been explored to improve system throughput [43]. While these approaches are synergistic to our approach, they either focus on optimizing secondary metrics, or use resource partitioning schemes not applicable to HPC-ML use cases and workloads.

X. CONCLUSION & FUTURE WORK

This paper addresses the inability to prioritize co-scheduled jobs on GPUs. We propose PACS and BCS policies that enable priority-based resource allocation in multi-tenant GPU environments. PACS allocates 100% GPU resources to high-priority HPC jobs while co-scheduling low-priority jobs with long-running workloads. Experimental results on a production GPU cluster demonstrate reduced turnaround time for high-priority jobs without preemption while maintaining bounded slowdowns for lower-priority workloads. Our analysis reveals strong correlation between application characteristics and co-scheduled LLM training throughput. The proposed policies are workload-agnostic and apply beyond the LLM training use case presented.

Future work includes: (1) extending to AMD GPUs and APUs leveraging open-source stacks for block-level resource control, (2) evaluating diverse job geometries, and (3) developing performance models for kernel-level and application-level slowdown prediction to improve bounded-perf policies.

ACKNOWLEDGMENTS

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344 (LNL-PROC-2016978).

REFERENCES

- [1] N. A. Simakov and et al, “A workload analysis of nsf’s innovative hpc resources using xdmmod,” 2018. [Online]. Available: <https://arxiv.org/abs/1801.04306>
- [2] H. Bhatia, H. I. Ingolfsson, F. Di Natale, J. R. Ghavez, J. Y. Moon, and U. N. N. S. Administration, “Multiscale machine-learned modeling infrastructure core,” 09 2021. [Online]. Available: <https://www.osti.gov/biblio/1835065>

- [3] D. H. Ahn and et al, "Scalable composition and analysis techniques for massive scientific workflows," in *2022 IEEE 18th International Conference on e-Science (e-Science)*, 2022, pp. 32–43.
- [4] H. Bhatia and et al, "Autonomous multiscale library," 05 2023. [Online]. Available: <https://www.osti.gov/biblio/1992083>
- [5] R. Beall, "Livchat....so far." Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States). [Online]. Available: <https://www.osti.gov/biblio/2377240>
- [6] Argonne National Laboratory, "AuroraGPT: Foundation Models for Science," <https://auroragpt.anl.gov>, 2025, accessed: 2025-05-06.
- [7] NVIDIA, "Nvidia multi-process service (mps)," 2024, accessed: 2025-02-23. [Online]. Available: <https://docs.nvidia.com/deploy/mps/index.html>
- [8] N. Corporation, "Nvidia multi-instance gpu (mig) user guide," 2024, accessed: 2025-02-23. [Online]. Available: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/index.html>
- [9] AMD, "Setting cus," 2025, accessed: 2025-02-28. [Online]. Available: <https://rocm.docs.amd.com/en/latest/how-to/setting-cus.html>
- [10] U. Saroliya, E. Arima, D. Liu, and M. Schulz, "Reinforcement learning-driven co-scheduling and diverse resource assignments on numa systems," in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. IEEE, 2024, pp. 170–178.
- [11] T. Allen, X. Feng, and R. Ge, "Slate: Enabling workload-aware efficient multiprocessing for modern gpgpus," 05 2019.
- [12] D. H. Ahn and et al, "Flux: Overcoming scheduling challenges for exascale workflows," *Future Generation Computer Systems*, vol. 110, pp. 202–213, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X19317169>
- [13] S. Pai, M. J. Thazhuthaveetil, and R. Govindarajan, "Improving gpgpu concurrency with elastic kernels," in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 407–418. [Online]. Available: <https://doi.org/10.1145/2451116.2451160>
- [14] J. Bakita and J. H. Anderson, "Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems," in *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*, vol. 335, Dagstuhl, Germany, 2025, pp. 21:1–21:25. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECRTS.2025.21>
- [15] X. Liu and et al, "Muxflow: efficient gpu sharing in production-level clusters with more than 10000 gpus," *Science China Information Sciences*, vol. 67, no. 12, p. 222101, 2024.
- [16] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, fine-grained gpu sharing for ml applications," ser. EuroSys '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1075–1092. [Online]. Available: <https://doi.org/10.1145/3627703.3629578>
- [17] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, "Transparent GPU sharing in container clouds for deep learning workloads," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 69–85. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/wu>
- [18] O. Pearce and et al, "Raja performance suite: Performance portability analysis with caliper and thicket," in *Proceedings of the SC '24 Workshop*, ser. SC-W '24. IEEE Press, 2025, p. 1206–1218. [Online]. Available: <https://doi.org/10.1109/SCW63240.2024.00162>
- [19] A. P. Thompson and et al, "LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales," *Comp. Phys. Comm.*, vol. 271, p. 108171, 2022.
- [20] G. Team, "Sw4lite - a seismic wave propagation code," 2025. [Online]. Available: <https://github.com/geodynamics/sw4lite>
- [21] V. A. Dobrev, T. V. Kolev, and R. N. Rieben, "High-order curvilinear finite element methods for lagrangian hydrodynamics," *SIAM Journal on Scientific Computing*, vol. 34, no. 5, pp. B606–B641, 2012. [Online]. Available: <https://doi.org/10.1137/120864672>
- [22] D. Richards, P. Brantley, S. Dawson, S. Mckenley, and M. O'Brien, "Quicksilver," 03 2016. [Online]. Available: <https://www.osti.gov/biblio/1313660>
- [23] A. J. Kunen, T. S. Bailey, and P. N. Brown, "Kripke - a massively parallel transport mini-app." Lawrence Livermore National Lab. (LLNL), Livermore, CA (United States), 06 2015. [Online]. Available: <https://www.osti.gov/biblio/1229802>
- [24] Y. Zheng and et al, "Llamafactory: Unified efficient fine-tuning of 100+ language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. [Online]. Available: <http://arxiv.org/abs/2403.13372>
- [25] A. G. et al., "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [26] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023, dataset available at <https://huggingface.co/datasets/tatsu-lab/alpaca>.
- [27] N. Corporation, *NVIDIA Nsight Systems User Guide*, 2025, accessed: 2025-02-28. [Online]. Available: <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>
- [28] J. McCalpin, "Memory bandwidth and machine balance in high performance computers," *IEEE Technical Committee on Computer Architecture Newsletter*, pp. 19–25, 12 1995.
- [29] N. Corporation, "Nvidia nsight systems," <https://developer.nvidia.com/nsight-systems>, 2025.
- [30] A. Maurya and et al, "Datastates-llm: Lazy asynchronous checkpointing for large language models," in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 227–239. [Online]. Available: <https://doi.org/10.1145/3625549.3658685>
- [31] B. Wan and et al, "Bytecheckpoint: A unified checkpointing system for large foundation model development," 2025. [Online]. Available: <https://arxiv.org/abs/2407.20143>
- [32] R. Bashizade, Y. Li, and A. R. Lebeck, "Adaptive simultaneous multi-tenancy for gpus," in *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer, 2018, pp. 83–106.
- [33] M. Pavlidakis, G. Vasiliadis, S. Mavridis, A. Argyros, A. Chazapis, and A. Bilas, "Guardian: Safe gpu sharing in multi-tenant environments," in *Proceedings of the 25th International Middleware Conference*, 2024, pp. 313–326.
- [34] J. Bakita and J. H. Anderson, "Hardware compute partitioning on nvidia gpus," in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2023, pp. 54–66.
- [35] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, fine-grained gpu sharing for ml applications," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 1075–1092.
- [36] S. Zhang and et al, "Efficient performance-aware gpu sharing with compatibility and isolation through kernel space interception," in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, 2025, pp. 1003–1019.
- [37] B. Zhang, S. Li, and Z. Li, "Miger: Integrating multi-instance gpu and multi-process service for deep learning clusters," in *Proceedings of the 53rd International Conference on Parallel Processing*, 2024, pp. 504–513.
- [38] A. Siavashi and M. Momtazpour, "A multi-objective framework for optimizing gpu-enabled vm placement in cloud data centers with multi-instance gpu technology," *arXiv preprint arXiv:2502.01909*, 2025.
- [39] W. Wu, "Fikit: Priority-based real-time gpu multi-tasking scheduling with kernel identification," *arXiv preprint arXiv:2311.10359*, 2023.
- [40] G. Gilman and R. J. Walls, "Refine: A reactive and fine-grained scheduling framework for concurrency on general purpose gpus," in *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, 2025, pp. 193–209.
- [41] A. F. Babaei and T. Chantem, "Daris: An oversubscribed spatio-temporal scheduler for real-time dnn inference on gpus," *arXiv preprint arXiv:2504.08795*, 2025.
- [42] Y. Luo and et al, "Scheduling deep learning jobs in multi-tenant gpu clusters via wise resource sharing," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. IEEE, 2024, pp. 1–10.
- [43] K. Wang and et al, "Concurrent analytical query processing with gpus," *Proceedings of the VLDB Endowment*, vol. 7, no. 11, pp. 1011–1022, 2014.